

Desenvolvimento Web 2

Aula 5

Orientação a Objetos em JavaScript Construtores, Protótipos e Herança

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Construtores
- Protótipos
 - Usando Protótipos com Construtores
 - Adicionando Métodos em Protótipos
- Herança
 - Métodos Herdados de `Object.prototype`
 - Herança entre Objetos
 - Herança de Construtores

Construtores

- Basicamente, um **construtor** é uma função usada com o operador **new** para criar um objeto. Objetos criados com um mesmo construtor têm as mesmas propriedades e métodos.
- Uma diferença com relação às funções comuns é que os nomes dos construtores devem começar com uma letra maiúscula. Exemplo:

```
function Pessoa() { }
```

- Após definir o construtor, é possível criar instâncias de objetos usando o operador **new**. Se não houver parâmetros para passar ao construtor, os parênteses podem ser omitidos.

```
var pessoa1 = new Pessoa();  
var pessoa2 = new Pessoa;  
console.log(pessoa1 instanceof Pessoa); // true  
console.log(pessoa2 instanceof Pessoa); // true
```

Aqui, o operador **instanceof** indica que os dois objetos são instâncias do tipo **Pessoa**.

Construtores

- O propósito de um construtor é facilitar a criação de objetos com as mesmas **propriedades de dados e métodos**. Sendo assim, é preciso adicioná-los ao construtor.

```
function Pessoa(nome) {  
  this.nome = nome;  
  this.retornaNome = function() {  
    console.log(this.nome);  
  };  
}
```

Nesse exemplo, foram adicionados a propriedade **nome** e o método **retornaNome()** no construtor **Pessoa**. Quando o construtor é chamado, o operador **new** cria automaticamente o objeto **this**, que corresponde a uma instância do tipo do construtor.

```
var pessoa1 = new Pessoa("Pedro");  
var pessoa2 = new Pessoa("Maria");  
console.log(pessoa1.nome); // "Pedro"  
console.log(pessoa2.nome); // "Maria"  
pessoa1.retornaNome();     // "Pedro"  
pessoa2.retornaNome();     // "Maria"
```

Com isso, é possível usar o construtor **Pessoa** para criar objetos com uma propriedade **nome** inicializada. Como cada objeto tem seu próprio valor para **nome**, o método **retornaNome()** retorna um valor diferente de acordo com o objeto que chamou o método.

This constructor function may be converted to a class declaration.

```
function Pessoa(nome: any): void
```

Quick Fix... (Ctrl+.)

```
function Pessoa(nome) {
```

No Visual Studio Code, para converter a sintaxe de construtor da **ES5** para **ES6**, passe o mouse sobre os três pontinhos e selecione **Quick Fix**.

```
class Pessoa {  
  constructor(nome) {  
    this.nome = nome;  
    this.retornaNome = function () {  
      console.log(this.nome);  
    };  
  }  
}
```

Construtores

- Os construtores permitem inicializar uma instância de um tipo de **maneira detalhada**, efetuando todas as configurações de propriedades necessárias antes que o objeto possa ser usado.

```
function Pessoa(nome) {  
  Object.defineProperty(this, "nome", {  
    get: function() {  
      return nome;  
    },  
    set: function(novoNome) {  
      nome = novoNome;  
    },  
    enumerable: true,  
    configurable: true  
  });  
  this.retornaNome = function() {  
    console.log(this.nome);  
  };  
}
```

Por exemplo, **Object.defineProperty()** pode ser usado em um construtor para configurar propriedades informando: o objeto (**this**), o nome de uma propriedade ("**nome**") e seus atributos (**get**, **set**, **enumerable**, **configurable**).

Ao usar construtores, as **propriedades** do objeto criado pertencem ao **tipo de referência** definido pelo construtor (**Pessoa**), e não ao **objeto em si** (**pessoa**) como no exemplo abaixo.

```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
}
```

Protótipos

- Os construtores permitem criar instâncias de objetos com as **mesmas propriedades e métodos**.
- Nos exemplos apresentados, cada instância possui o mesmo método **retornaNome()**. Isso significa que, se houver cem instâncias do objeto **Pessoa**, haverá cem cópias deste método.
- Seria muito mais eficiente se todas as instâncias compartilhassem um **mesmo método** e esse método pudesse usar **this** para acessar o dado apropriado. É nesse caso que os **protótipos** entram em cena.

```
function Pessoa(nome) {  
    this.nome = nome;  
    this.retornaNome = function() {  
        console.log(this.nome);  
    };  
}
```

```
var pessoa1 = new Pessoa("Pedro");  
var pessoa2 = new Pessoa("Maria");  
pessoa1.retornaNome(); // "Pedro"  
pessoa2.retornaNome(); // "Maria"
```

*Um **protótipo** é como uma receita para um objeto. Quase todos os **métodos** (exceto alguns métodos nativos) estão associadas a uma propriedade **prototype** do objeto, que é usada durante a criação de novas instâncias e faz com que o método seja compartilhado entre todas as suas instâncias.*

Protótipos

- Um exemplo de método associado a um protótipo é o **hasOwnProperty()** definido no protótipo do tipo **Object**.

```
var livro = {  
  titulo: "Crime e Castigo"  
};  
console.log("hasOwnProperty" in livro); // true  
console.log(livro.hasOwnProperty("hasOwnProperty")); // false (não é uma propriedade própria)  
console.log(Object.prototype.hasOwnProperty("hasOwnProperty")); // true
```

Por padrão, objetos literais estão associados a **Object.prototype**. Com isso, embora não exista a definição de **hasOwnProperty()** no objeto **livro**, esse método pode ser acessado pelo objeto, pois sua definição existe em **Object.prototype**.

- Também é possível identificar se uma propriedade foi herdada de um protótipo usando uma função como esta:

```
function estaAssociadaAPrototipoHerdado(objeto, nome) {  
  // Se a propriedade estiver no objeto e não for uma propriedade própria, retorna true.  
  return nome in objeto && !objeto.hasOwnProperty(nome);  
}  
console.log(estaAssociadaAPrototipoHerdado(livro, "titulo")); // false (está no objeto, mas é própria)  
console.log(estaAssociadaAPrototipoHerdado(livro, "hasOwnProperty")); // true
```

Protótipos

- Em JavaScript, o acesso a uma **propriedade** é feito da seguinte forma:
 1. Ao ler uma **propriedade** em um objeto, inicialmente a linguagem procura uma **propriedade própria** com o mesmo nome.
 2. Se a propriedade for encontrada no objeto, o valor da propriedade é retornado.
 3. Se não for encontrada, a linguagem busca uma **propriedade de protótipo** com o mesmo nome. Caso exista, seu valor é retornado.
 4. Se a busca terminar sem que uma propriedade com o nome definido tenha sido encontrada, é retornado **undefined**.

Protótipos

- Protótipos permitem a sobrescrita de suas propriedades, de modo que uma **propriedade do protótipo** se torne uma **propriedade própria**.

Nesse exemplo, é criado um objeto sem nenhuma propriedade própria, ou seja, um objeto que tem apenas as propriedades do seu protótipo (**Object.prototype**).

O método **toString()** é uma propriedade de protótipo de **Object.prototype**, que retorna por padrão "[object Object]".

Ao definir uma **propriedade própria** de mesmo nome, essa propriedade será usada sempre que **toString()** for chamada no objeto, ou seja, a **propriedade própria** irá sobrepor a **propriedade do protótipo**.

```
var objeto = {};  
  
console.log(objeto.toString()); // "[object Object]" (toString do protótipo)  
objeto.toString = function() { // Criação do toString próprio  
    return "Propriedade própria";  
};  
  
console.log(objeto.toString()); // "Propriedade própria" (toString próprio)  
delete objeto.toString; // Apaga o toString próprio  
  
console.log(objeto.toString()); // "[object Object]" (toString do protótipo)  
delete objeto.toString; // Sem efeito (delete só funciona em propriedades próprias)
```

Usando Protótipos com Construtores

- Como vimos, ao definir um método no construtor, será criada uma cópia deste método para cada instância de objeto. Logo, é interessante incluir os métodos no **protótipo** do objeto, de modo que cada método seja definido somente uma vez para todas as instâncias.

```
function Pessoa(nome) {  
    this.nome = nome;  
}  
Pessoa.prototype.retornaNome = function() {  
    console.log(this.nome);  
};  
var pessoa1 = new Pessoa("Pedro");  
var pessoa2 = new Pessoa("Maria");  
pessoa1.retornaNome(); // "Pedro"  
pessoa2.retornaNome(); // "Maria"
```

Nessa versão do construtor **Pessoa**, o método **retornaNome()** está definido no protótipo, e não no construtor. Isso não altera em nada o comportamento do método. A única diferença é que não haverá mais uma cópia do método para cada instância do objeto **Pessoa**.

Usando Protótipos com Construtores

- Embora seja possível adicionar propriedades ao protótipo, uma a uma, existe uma forma mais eficiente que consiste em descrever o **protótipo** usando um **objeto literal** e a propriedade **constructor** do protótipo.

A definição da propriedade **constructor** no protótipo é necessária para que ela seja vinculada ao tipo **Pessoa**. Por isso, para não esquecer de incluí-la, uma boa prática é defini-la sempre como a primeira propriedade do protótipo.

```
function Pessoa(nome)
  this.nome = nome;
}
Pessoa.prototype = {
  constructor: Pessoa,
  retornaNome: function() {
    console.log(this.nome);
  },
  toString: function() {
    return "Pessoa: " + this.nome;
  }
};
var pessoa1 = new Pessoa("Pedro");
var pessoa2 = new Pessoa("Maria");
pessoa1.retornaNome();           // "Pedro"
pessoa2.retornaNome();           // "Maria"
console.log(pessoa1.toString()); // "Pessoa: Pedro"
console.log(pessoa2.toString()); // "Pessoa: Maria"
```

Esse código define dois métodos no protótipo: **retornaNome()** e **toString()**, eliminando a necessidade de digitar **Pessoa.prototype** mais de uma vez.

Adicionando Métodos em Protótipos

- Os protótipos permitem adicionar novos métodos a qualquer momento, os quais passam a ficar acessíveis para as instâncias do objeto.

```
function Pessoa(nome) {  
    this.nome = nome;  
}  
Pessoa.prototype = {  
    constructor: Pessoa,  
    retornaNome: function() {  
        console.log(this.nome);  
    },  
    toString: function() {  
        return "Pessoa: " + this.nome;  
    }  
};  
var pessoa1 = new Pessoa("Pedro");  
var pessoa2 = new Pessoa("Maria");  
Pessoa.prototype.retornaOla = function() { // Adiciona um novo método  
    console.log("Olá");  
};  
pessoa1.retornaOla(); // "Olá"  
pessoa2.retornaOla(); // "Olá"
```

Nesse exemplo, o tipo **Pessoa** começa com apenas dois métodos: **retornaNome()** e **toString()**. Após criar duas instâncias de **Pessoa**, é adicionado ao protótipo o método **retornaOla()**, que fica disponível para ambas as instâncias.

Adicionando Métodos em Protótipos

- Também é possível adicionar métodos a protótipos de **objetos nativos** da JavaScript. Por exemplo, para adicionar um novo método que possa ser usado em todos os *arrays*, basta modificar **Array.prototype**:

```
Array.prototype.somar = function() {  
    return this.reduce(function(anterior, atual) {  
        return anterior + atual;  
    });  
};  
var numeros = [1, 2, 3, 4];  
var resultado = numeros.somar();  
console.log(resultado); // 10
```

Aqui, **this** se refere a **numeros** (instância de **Array**). Logo, **numeros** consegue usar outros métodos de **Array**, como **reduce()**, que faz a somatória de elementos de um *array*.

Nesse exemplo, é adicionado o método **somar()** ao **Array.prototype**. Este método executa e retorna o resultado do método **reduce()** existente no tipo **Array**. Com isso, **numeros**, que é uma instância de **Array**, passa a ter acesso ao método **somar()**.

*Embora pareça interessante adicionar métodos a protótipos de **objetos nativos**, a fim de criar novas funcionalidades, essa pode não ser uma boa ideia, pois os desenvolvedores esperam determinados métodos dos objetos nativos e podem desconhecer o comportamento de métodos criados por outros desenvolvedores.*

Herança

- Em JavaScript, a **herança** pode ocorrer entre objetos por meio dos **protótipos**, sem a necessidade de existir uma **estrutura de classes** para definir uma relação entre os objetos.
- O próprio fato de as **propriedades dos protótipos** estarem disponíveis nas **instâncias** dos objetos já é uma forma de **herança**.
- Um exemplo simples da herança em JavaScript é que, por padrão, todos os objetos, incluindo aqueles definidos pelo desenvolvedor, automaticamente herdam de **Object.prototype**.

```
var livro = {  
  title: "Crime e Castigo"  
}
```

```
var prototipo = Object.getPrototypeOf(livro); // Recupera o protótipo de um objeto  
console.log(prototipo === Object.prototype); // true
```

Nesse exemplo, **livro** tem um protótipo igual a **Object.prototype**, ou seja, este objeto herda automaticamente os **métodos de Object.prototype**.

Métodos Herdados de Object.prototype

- Dentre os vários métodos nativos herdados de **Object.prototype**, dois deles se destacam quando é preciso que os objetos funcionem de maneira customizada: **valueOf()** e **toString()**.
- O método **valueOf()** é chamado sempre que um **operador** é usado em um objeto. Por padrão, ele retorna a **instância** do objeto.
- Porém, os **tipos primitivos** sobrescrevem **valueOf()** de modo que uma string é retornada como **String**, um número como **Number** etc.

*No caso do tipo **Date**, o método **valueOf()** foi nativamente sobrescrito para retornar uma data/hora em milissegundos, permitindo, por exemplo, que duas datas sejam comparadas.*

```
var agora = new Date();  
var antes = new Date(2020, 0, 1);  
console.log(agora > antes); // true
```

Nesse exemplo, o objeto **agora** é um **Date** que representa a data atual, e o objeto **antes** é uma data no passado. Quando o operador **maior que (>)** é usado para comparar as datas, o método **valueOf()** é chamado em ambos os objetos antes de a comparação ser executada.

Métodos Herdados de Object.prototype

- O método **toString()** retorna uma representação do objeto em forma de string e é chamado sempre que **valueOf()** retornar uma **instância** de objeto.
- Por exemplo, ao fazer a adição de uma **string** à um **objeto**, o método **valueOf()** será chamado para ambos os operandos na adição. Porém, **toString()** será chamado apenas para o **objeto** e retornará um valor padrão definido em **Object.prototype**.

*Em algumas situações, o método **toString()** também será chamado implicitamente para **valores primitivos**. Por exemplo, quando se faz a operação de adição de uma **string** a outro tipo de **valor primitivo (Number, Boolean etc)**, este valor é automaticamente convertido para string (23 se torna "23", **true** se torna "**true**").*

```
console.log("O resultado é " + 23);  
console.log("2 + 2 = 4 -> " + true);
```

```
O resultado é 23  
2 + 2 = 4 -> true
```


Métodos Herdados de Object.prototype

```
var livro = {  
  title: "Crime e Castigo"  
};  
var mensagem = "Livro = " + livro;  
console.log(mensagem); // "Livro = [object Object]"  
  
var livro = {  
  title: "Crime e Castigo",  
  toString: function() {  
    return this.title  
  }  
};  
var mensagem = "Livro = " + livro;  
console.log(mensagem); // "Livro = Crime e Castigo"
```

Esse código forma uma string por meio da adição da string **"Livro = "** com o objeto **livro**. Ao usar operador **+**, os métodos **valueOf()** e **toString()** do objeto **livro** são chamados, retornando o valor padrão **"[object Object]"** herdado de **Object.prototype**. Nesse caso, é mais interessante definir um método **toString()** próprio, para que ele retorne uma informação útil.

Aqui, o método **toString()** é customizado para que retorne o **título** do livro.

Herança entre Objetos

- Para implementar a **herança entre objetos**, basta especificar qual protótipo de objeto deve ser herdado para o novo objeto, o que é feito por meio do método **Object.create()**.
- Nesse exemplo, **Object.prototype** é definido explicitamente para o objeto **livro** por meio deste método.

```
var livro = {  
  titulo: "Crime e Castigo"  
};  
var livro = Object.create(Object.prototype, {  
  titulo: {  
    configurable: true,  
    enumerable: true,  
    writable: true,  
    value: "Crime e Castigo"  
  }  
});
```

O método **Object.create()** aceita dois argumentos. O **1º argumento** é o protótipo de objeto que será herdado ao novo objeto. O **2º argumento (opcional)** é um objeto contendo descritores da propriedade **titulo**, no mesmo formato usado em **Object.defineProperty()**.

*Nesse código, as duas declarações do objeto **livro** são equivalentes, de modo que os objetos resultantes têm exatamente as mesmas características. Logo, é pouco provável que algum desenvolvedor vá escrever código que herde explicitamente de **Object.prototype**.*

Herança entre Objetos

- Porém, pode ser bastante útil usar o método **Object.create()** para herdar protótipos de outros tipos (diferentes de **Object**).

```
var pessoa1 = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
};  
var pessoa2 = Object.create(pessoa1, {  
  nome: {  
    configurable: true,  
    enumerable: true,  
    writable: true,  
    value: "Maria"  
  }  
});
```

```
pessoa1.retornaNome(); // "Pedro"  
pessoa2.retornaNome(); // "Maria"  
console.log(pessoa1.hasOwnProperty("retornaNome")); // true  
console.log(pessoa2.hasOwnProperty("retornaNome")); // false  
// Verifica se um objeto usa o protótipo de outro:  
console.log(pessoa1.isPrototypeOf(pessoa2)); // true
```

Esse código cria um objeto **pessoa1** com uma propriedade **nome** e um método **retornaNome()**. Em seguida, o objeto **pessoa2** herda **nome** e **retornaNome()** de **pessoa1**.

Porém, **pessoa2** é definido por meio de **Object.create()**, que também define uma propriedade própria chamada **nome**. Essa propriedade própria sobrepõe a propriedade **nome** do protótipo de **pessoa1**.

Com isso, **pessoa1.retornaNome()** exibe "Pedro", enquanto **pessoa2.retornaNome()** exibe "Maria". Note que o método **retornaNome()** é próprio somente em **pessoa1**, já que **pessoa2** está apenas herdando este método.

Resumindo, **pessoa2** está herdando do protótipo do objeto **pessoa1**, e **pessoa1** de **Object.prototype**.

Herança de Construtores

- Ao criar um construtor para um novo tipo de referência, a propriedade **prototype** é definida internamente para conter um novo objeto genérico que herda de **Object.prototype** e que tem uma única propriedade própria chamada **constructor**.

*Esses dois códigos são equivalentes. Ambos definem a propriedade **prototype** do construtor com um objeto que herda de **Object.prototype**, o que significa que qualquer instância de **MeuConstrutor** também herdará de **Object.prototype**. No entanto, é possível sobrepor a propriedade **prototype** do tipo definido no construtor por meio da **herança de construtores**.*

```
// Ao escrever isto:
function MeuConstrutor() {
    // inicialização
}

// A engine do JavaScript faz isto internamente:
MeuConstrutor.prototype = Object.create(Object.prototype, {
  constructor: {
    configurable: true,
    enumerable: true,
    writable: true,
    value: MeuConstrutor
  }
});
```

Herança de Construtores

```
function Retangulo(comprimento, largura) { ←
  this.comprimento = comprimento;
  this.largura = largura;
}
Retangulo.prototype.getArea = function() {
  return this.comprimento * this.largura;
};
function Quadrado(tamanho) { ←
  this.comprimento = tamanho;
  this.largura = tamanho;
}
Quadrado.prototype = Object.create(Retangulo.prototype, {
  constructor: {
    configurable: true,
    enumerable: true,
    writable: true,
    value: Quadrado
  }
});
var ret = new Retangulo(5, 10);
console.log(ret.getArea()); // 50
var quad = new Quadrado(6);
console.log(quad.getArea()); // 36
console.log(ret instanceof Retangulo); // true
console.log(ret instanceof Quadrado); // false
console.log(quad instanceof Quadrado); // true
console.log(quad instanceof Retangulo); // true
```

Nesse código, existem dois construtores: **Retangulo** e **Quadrado**. Aqui, a herança de construtor sobrescreve **Quadrado.prototype** pelo protótipo herdado de **Retangulo.prototype**.

Aqui, a variável **ret** é criada como uma instância de **Retangulo**, e **quad** como uma instância de **Quadrado**. Ambos os objetos têm o método **getArea()** porque ele é herdado de **Retangulo.prototype**.

Note que, enquanto a variável **ret** é considerada uma instância apenas de **Retangulo**, a variável **quad** é considerada uma instância de **Quadrado** e **Retangulo**.

Referências

- ZAKAS, Nicholas C. **Princípios de Orientação a Objetos em JavaScript**. São Paulo: Editora Novatec, 2014.